

claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_3512746c-54e\agent-aa39282a37f4ff850.jsonl`
- SHA-256: `363204b7f1ce5e403477bfd42fa0e9e6fc041e6968bf0fbfd90c8fdd40b7a20c`
- Source modified: `2026-07-06T09:12:50+00:00`
- Imported at: `2026-07-08T16:00:05+00:00`
- project: `wf_3512746c-54e`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`
```

Transcript

1. user (2026-07-06T09:09:16.742Z)

Reviewing GitHub PR #39 of the repo at C:/Users/avidu/Projects/Telegram (branch feat/track-e-selective-scan -> main, one commit db87cc0).

It implements ADR 0009 Track E: selective source scanning ? /watch, /unwatch, /channels, /watchlist commands +

an ingestion gate in telethon_client.py that skips unwatched sources. Changed files ONLY: command_service.py (new /watch etc + _resolve_source), commands.py (re-export patterns), db.py (set_source_watched, list_account_sources), telethon_client.py (the gate), tests/test_command_service.py (10 tests).

Diff +254/-3. READ docs/adr/0009-selective-channel-scanning.md ? it is the spec this must conform to.

KEY ADR 0009 REQUIREMENTS (quote/verify against the code):

- Line 37-38: "The default is locked as opt-in: DEFAULT 0, so a freshly-onboarded user scans nothing until they opt in."
- Line 44: "...return early if watched is false, before any dedup or filter work."
- Line 130-131: "New channels discovered after the migration default to watched = 0 and must be explicitly watched."
- Line 48-54 (discoverability): lazy-populated rows should be created watched=0 so they appear in /channels for opt-in;
/channels should also upsert candidates from client.get_dialogs().
- Line 64: "/watch <#name|id>" ? resolve by channel name/username OR id.

Ground rules:

- READ actual code (Read/Grep), cite file:line. You MAY run repros with C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe (build schema via db._apply_schema on aiosqlite ":memory:"); seed via db.upsert_account + db.upsert_source; drive command_service.CommandService().handle()).
- NOTE: real telegram sources have a NUMERIC provider_source_id (str(channel_id) e.g. '-1001234567890'); upsert_channel creates them with watched=True. The on_new_message gate is inside a '# pragma: no cover' function.
- Authoritative gate ALREADY RUN (don't re-run full suite): ruff check + format clean; pytest 492 passed @ 97.10% (floor 80%).
- Report ONLY defects in THIS PR's diff. Deviation from the ACCEPTED ADR 0009 is a high-value finding. No praise.
Concrete failure scenario per finding, ranked most-severe first, honest severity.
- Severity: blocker (feature broken / data loss on the live path), high (real bug or ADR-contract violation users hit),
medium (edge/latent/half-broken feature), low (hygiene/UX/discoverability).

ADVERSARIALLY VERIFY this finding ? try to REFUTE it. Read the cited code AND docs/adr/0009-selective-channel-scanning.md, trace real control flow, run a repro with

C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe if useful. CONFIRMED only if the defect genuinely ships in THIS PR's diff and (for ADR-conformance) actually deviates from the accepted ADR. REFUTED if the code conforms / trigger can't occur / pre-existing. PLAUSIBLE if unsettled. Corrected severity.

FINDING (tests_ux): New channels are auto-watched on first message, violating ADR 0009 opt-in default (DEFAULT 0)

severity blocker | src/tg_video_filter/telethon_client.py:189

summary: The watched-gate only skips channels that ALREADY have a source row with watched=false; a brand-new channel (no row) falls through the gate and is then lazily created with watched=True by db.upsert_channel, so its first message is ingested AND delivered.

scenario: A freshly-onboarded user has watched nothing. A video arrives from channel -1005 that has never been seen before. At telethon_client.py:189 get_source_by_provider_id returns None, so `if src is not None and not src.watched` is False and the gate does NOT return. Ingestion proceeds through dedup, then telethon_client.py:249 calls db.upsert_channel(...) which hard-codes watched=True (db.py:1402). The very first message is evaluated, inserted, and delivered to the user's target. Every new channel the account is a member of thus gets scanned and delivered exactly once on first traffic and permanently thereafter -- the exact 'scan everything' behavior ADR 0009 set out to eliminate. This directly contradicts ADR 0009 line 37-38 ('DEFAULT 0, so a freshly-onboarded user scans nothing until they explicitly /watch') and line 130-131 ('New channels discovered after the migration default to watched = 0 and must be explicitly watched').

evidence: Repro confirmed: `db.upsert\_channel(c, Channel(user\_id=111, channel\_id=-1005, title='Brand New'))` then reading the source back yields `watched = True`. The shim at db.py:1402 sets `watched=True` # back-compat: pre-ADR-0011 channels were all watched. The gate at telethon_client.py:190 (`if src is not None and not src.watched: return`) has no branch for the src-is-None case, so new sources are never blocked. Comment at telethon_client.py:186-188 explicitly acknowledges 'if absent, we let the legacy path upsert it with watched=True for backwards-compatibility' -- but that back-compat rule applies only to the migration backfill (UPDATE ... WHERE seen_first_at IS NOT NULL, ADR line 126), NOT to channels discovered AFTER the migration, which ADR line 130-131 requires to default to watched=0.

rec: The gate must treat an absent source row as unwatched (skip), and lazy population must create rows with watched=0. Change the gate to `if src is None or not src.watched: return` (skipping unseen channels), and make the lazy source-creation on the ingest path create the row with watched=0 (do not route through upsert_channel's watched=True shim) so a newly-seen channel is recorded for /channels discoverability but not scanned until /watch.

2. assistant (2026-07-06T09:09:19.312Z)

I'll verify this finding by reading the cited code and the ADR, then running a repro.

3. user (2026-07-06T09:09:20.332Z)

```

1      # ADR 0009: Selective channel scanning and multiplexed routing
2
3      - **Status**: Accepted
4      - **Date**: 2026-07-04
5      - **Decision owner**: project owner
6
7      ## Context
8
9      Today the userbot scans every channel/group the account is a member of.
10     `manager.py` registers `events.NewMessage(incoming=True)` with no `chats=`
11     filter, and `on_new_message` processes every video regardless of source. The
12     `channels` table exists but is populated lazily as a record of seen channels;
13     it is not used as a filter.
14
15     The product vision is a multiplexing video re-distribution system:
16
17     - A small set of high-quality source channels (e.g. spiritual, classical
18       music) are scanned for incoming videos.
19     - Matches are routed to themed destination broadcast channels (e.g. a
20       Reggae channel, a Rap channel) that subscribers opt into.
21     - A user therefore subscribes to outputs by topic, not to inputs by source.
22
23     This requires two capabilities the current architecture does not have:

```

```

24
25 1. **Selective scanning** ? choose which source channels are watched, rather
26   than ingesting all of them.
27 2. **Per-source ? per-destination routing** ? a match from source channel X is
28   delivered to a specific destination channel Y, instead of the single global
29   `delivery_target` (ADR 0005).
30
31 ## Decision
32
33 Add three additive, independently-shippable capabilities:
34
35 ### 1. Selective scanning (watched-flag)
36
37 Add a `watched` flag to the `channels` table. The default is locked as
38 **opt-in: `DEFAULT 0`**, so a freshly-onboarded user scans nothing until they
39 explicitly `/watch` a channel. This matches the multiplexing product shape
40 (sources are a *curated* set) and avoids blindly re-ingesting every chat the
41 account is a member of.
42
43 Gate `on_new_message` at the top: after computing `channel_id`, look up the
44 row and `return` early if `watched` is false, before any dedup or filter work.
45
46 This is a small migration plus ~5 lines in `telethon_client.py`.
47
48 **Discoverability note.** With `DEFAULT 0` plus the current lazy population
49 (a `channels` row is only inserted when a message is first seen), opt-in is
50 necessary but not sufficient for discoverability: a channel the account has
51 never received a message from has no row to flag. The `/channels` command
52 (step 2 below) therefore upserts candidate rows from `client.get_dialogs()`
53 so users can browse and watch channels they have not yet received traffic
54 from. The flag default and the enumeration flow are orthogonal ? both are
55 required.
56
57 ### 2. Channel enumeration and management commands
58
59 New Saved Messages commands (and, via ADR 0008, bot commands) building on the
60 existing `commands.py` pattern:
61
62 - `/channels` ? list the user's channels (via `client.get_dialogs()`), already
63   used in `backfill.py`) with their watched state.
64 - `/watch <#name|id>` ? set `watched=1` for a source channel.
65 - `/unwatch <#name|id>` ? set `watched=0`.
66 - `/watchlist` ? list only watched sources.
67
68 ### 3. Per-source ? per-destination routing rules
69
70 Introduce a `routing_rules` table (the canonical name; issue #25's `routes`
71 refers to the same concept and should be aligned to `routing_rules`), e.g.:
72
73 ```
74 routing_rules (
75     user_id          INTEGER NOT NULL REFERENCES users(id),
76     source_channel_id INTEGER NOT NULL,
77     dest_channel_id   INTEGER NOT NULL,    -- a broadcast channel / @username
78     filter_config_id  INTEGER,             -- optional per-route filter override
79     PRIMARY KEY (user_id, source_channel_id, dest_channel_id)
80 )
81 ```
82
83 On a match, look up applicable routing rules by `source_channel_id` and
84 deliver to each matching destination. The delivery layer (`delivery.py`)
85 already accepts a `target` per call, so this generalises the existing single
86 `delivery_target` to a per-match lookup. Users with no routing rules fall
87 back to the current global `delivery_target` behaviour (ADR 0005), preserving
88 backwards compatibility.

```

```

89
90 ## Consequences
91
92 **Positive**
93 - Enables the multiplexing product shape: scan curated sources, publish to
94   topical broadcast channels, let subscribers pick outputs by interest.
95 - Each capability is additive and independently deployable; none disturbs the
96   existing FORWARD/LINK/BOTH paths or the metadata-first ingestion (ADR 0003).
97 - Dedup (ADR 0006) still holds: Tier 1 (`document_id`) keys the *original*
98   file, so a routed re-upload is not re-processed.
99 - Channel enumeration reuses `client.get_dialogs()` already present in
100   `backfill.py`; no new Telethon capability is required.
101
102 **Negative**
103 - New schema (`channels.watched`, `routing_rules`) with migrations.
104 - **Routing loop invariant.** With multiplexed routing, the userbot account
105   may be a member of a *destination* broadcast channel and re-see its own
106   output, producing an infinite forward loop. The contract is: **a channel
107   is either a source or a destination, never both.** This is enforced in
108   `on_new_message` by excluding any channel that is a configured destination
109   (`source_channel_id IN (SELECT dest_channel_id FROM routing_rules WHERE
110   user_id = ?)`) before the watched-flag check. Issue #25 Track E implements
111   this; the invariant is stated here so the handler has a contract to code
112   against.
113 - Per-route filter overrides add config complexity; v1 may ship with global
114   filters only and a single implicit route (`source ? global target`), then
115   layer explicit per-route rules later.
116
117 ## Backwards compatibility
118
119 Adding `channels.watched` with `DEFAULT 0` would silently stop every existing
120 install from receiving matches until the user re-watches their channels ? a
121 behaviour change disguised as a schema change. To preserve the current
122 "ingest everything" semantics for existing installs, the migration backfills:
123
124 ```sql
125 ALTER TABLE channels ADD COLUMN watched INTEGER NOT NULL DEFAULT 0;
126 UPDATE channels SET watched = 1 WHERE seen_first_at IS NOT NULL;
127 ```
128
129 i.e. any channel the install has *already* seen traffic from is implicitly
130 watched, so pre-migration users observe no change. New channels discovered
131 after the migration default to `watched = 0` and must be explicitly watched ?
132 which is the intended new UX. This mirrors the "old rows behave exactly as
133 before" pattern established by ADR 0005.
134
135 ## Rollout order
136
137 1. **Selective scanning** (`channels.watched` + `/watch`, `/unwatch`,
138   `/channels`). Smallest unit; immediately useful even without routing.
139 2. **Named destination channels** ? extend `/set delivery_target` to allow
140   switching among a user-defined set, not just one global value.
141 3. **Per-source ? per-destination routing rules** (`routing_rules` table +
142   `/route <source> <dest>` command). The full multiplexing vision.
143
144 Each step is shippable and useful on its own; step 3 is not required to
145 deliver value from steps 1?2.
146
147 ## Alternatives considered
148
149 - **Server-side `chats=` filter at handler registration:** rejected ? the
150   watched set is per-user and mutable at runtime; re-registering Telethon
151   handlers on every `/watch` change is brittle. An in-handler lookup against
152   the `channels` table is simpler and hot-swappable.
153 - **Content-based classification (auto-detect genre):** explicitly out of

```

```

154     scope for v1. Routing is user-defined by source channel; automatic topic
155     detection could be a future enhancement but is not required to deliver the
156     multiplexing UX.
157
158     ## References
159
160     - ADR 0001 (userbot ingestion) ? the capability this selectively constrains.
161     - ADR 0005 (per-user delivery target) ? the single-target delivery this
162       generalises into per-route targets.
163     - ADR 0006 (content dedup) ? still applies; routed re-uploads are keyed by
164       the original `document_id`.
165     - ADR 0008 (bot API onboarding) ? the front-door that exposes `/watch`,
166       `/unwatch`, and routing commands to public users.
167

```

4. user (2026-07-06T09:09:20.997Z)

```

160         plus the existing (channel, message) instance dedup,
161         3. runs the filter engine (read from ``engine_holder`` so config
162           changes via ``/set`` take effect immediately),
163         4. inserts a VideoRecord row,
164         5. delivers a match to ``delivery_target`` (see ADR 0005).
165     """
166
167     async def on_new_message(event: NewMessage.Event) -> None: # pragma: no cover
168         message: Message = event.message # type: ignore[assignment]
169         if message is None:
170             return # Telethon edge case: event has no message (shouldn't happen)
171         # Only process genuine channel/group posts and private messages we
172         # receive ? outgoing messages (sent by our own account) are ignored.
173         if getattr(message, "out", False):
174             return
175
176         meta = extract_video_meta(message)
177         if meta is None:
178             return # not a video
179
180         channel_id = _peer_id(message.peer_id)
181         message_id = message.id
182         document_id = _extract_document_id(message)
183
184         # --- watched gate (ADR 0009, Track E) -----
185         # Skip ingestion for unwatched sources. The channel must already be in
186         # sources (lazy-created by the /channels enumeration or the legacy
187         # upsert_channel below); if absent, we let the legacy path upsert it
188         # with watched=True for backwards-compatibility.
189         src = await db.get_source_by_provider_id(conn, user.id, "telegram",
190 str(channel_id))
191         if src is not None and not src.watched:
192             return # silently skip unwatched source
193
194         # --- three-tier dedup (cheapest first) ---
195
196         # Tier 0: same (user, channel, message) instance ? already processed.
197         existing = await db.get_video(conn, user.id, channel_id, message_id)
198         if existing is not None:
199             return
200
201         # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
202         if document_id is not None:
203             existing = await db.get_video_by_document_id(conn, user.id, document_id)
204             if existing is not None:
205                 log.info(
206                     "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
207                     "original_channel=%s original_msg=%s",

```

```

207         user.id,
208         document_id,
209         channel_id,
210         message_id,
211         existing.channel_id,
212         existing.message_id,
213     )
214     return
215
216 # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
217 if (
218     meta.size_bytes is not None
219     and meta.duration_s is not None
220     and meta.height is not None
221     and meta.width is not None
222 ):
223     existing = await db.get_video_by_content_fingerprint(
224         conn,
225         user.id,
226         meta.size_bytes,
227         meta.duration_s,
228         meta.height,
229         meta.width,
230     )
231     if existing is not None:
232         log.info(
233             "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
234             "channel=%s msg=%s original_channel=%s original_msg=%s",
235             user.id,
236             meta.size_bytes,
237             meta.duration_s,
238             meta.height,
239             meta.width,
240             channel_id,
241             message_id,
242             existing.channel_id,
243             existing.message_id,
244         )
245     return
246
247 # Lazily record the channel (best-effort; failures are non-fatal).
248 try:
249     await db.upsert_channel(
250         conn,
251         Channel(user_id=user.id, channel_id=channel_id,
252 title=_channel_title(event)),
253     )
254 except Exception:
255     log.warning(
256         "failed to upsert channel %s for user %s", channel_id, user.id,
257 exc_info=True
258     )
259
260 result = engine_holder[ENGINE_KEY].evaluate(meta)
261 record = VideoRecord(
262     user_id=user.id,
263     channel_id=channel_id,
264     message_id=message_id,
265     document_id=document_id,
266     duration_s=meta.duration_s,
267     width=meta.width,
268     height=meta.height,
269     size_bytes=meta.size_bytes,
270     mime_type=meta.mime_type,
271     matched=result.matched,

```

5. assistant (2026-07-06T09:09:25.150Z)

Now let me look at `get\_source\_by\_provider\_id` and `upsert\_channel` to verify the mechanics.

6. user (2026-07-06T09:09:26.764Z)

```
75:     matched      INTEGER NOT NULL DEFAULT 0,
76:     forwarded     INTEGER NOT NULL DEFAULT 0,
99:     watched       INTEGER NOT NULL DEFAULT 0,
122:    matched        INTEGER NOT NULL DEFAULT 0,
133:    is_default      INTEGER NOT NULL DEFAULT 0,
150:    is_output       INTEGER NOT NULL DEFAULT 0,
162:    priority        INTEGER NOT NULL DEFAULT 0,
173:    attempt_count   INTEGER NOT NULL DEFAULT 0,
462:                matched      INTEGER NOT NULL DEFAULT 0,
463:                forwarded    INTEGER NOT NULL DEFAULT 0,
1104:async def upsert_source(conn: aiosqlite.Connection, source: Source) -> Source:
1142:async def get_source_by_provider_id(
1387:async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1402:                watched=True, # back-compat: pre-ADR-0011 channels were all watched
1537:async def set_source_watched(
1556:        query += " AND watched = 1"
1578:        watched=True,
```

7. user (2026-07-06T09:09:31.185Z)

```
1387     async def upsert_channel(conn: aiosqlite.Connection, channel: Channel) -> Channel:
1388         """Deprecated shim: upsert the :class:`Source` implied by a legacy Channel.
1389
1390         ADR 0011: ``channels`` is renamed to ``channels_deprecated``; this function
1391         reads/writes ``sources`` instead and maps the legacy ``Channel`` shape back.
1392         New code should call :func:`upsert_source` directly.
1393         """
1394         src = await upsert_source(
1395             conn,
1396             Source(
1397                 account_id=channel.user_id,
1398                 provider="telegram",
1399                 provider_source_id=str(channel.channel_id),
1400                 title=channel.title,
1401                 kind="channel",
1402                 watched=True, # back-compat: pre-ADR-0011 channels were all watched
1403                 seen_first_at=channel.seen_first_at,
1404             ),
1405         )
1406         return _source_to_channel(src)
1407
1408
1409     async def get_channel(conn: aiosqlite.Connection, user_id: int, channel_id: int) ->
Channel | None:
1410         """Deprecated shim: project the :class:`Source` row back to a legacy Channel."""
1411         src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1412         return _source_to_channel(src) if src else None
1413
1414
1415     def _source_to_channel(src: Source) -> Channel:
1416         """Project a provider-neutral :class:`Source` onto the legacy :class:`Channel`.
1417
1418         Only valid for Telegram sources (``provider == "telegram"``); the legacy
1419         ``Channel.channel_id`` is a Telegram-specific int id. Non-Telegram sources
1420         raise ``ValueError`` ? new code should use :class:`Source` directly instead
1421         of the deprecated :class:`Channel` shim (ADR 0011 provider-neutrality guard).
1422         """
1423         if src.provider != "telegram":
1424             raise ValueError(
```

```
1425         f"Cannot project {src.provider!r} source onto the Telegram-specific "
1426         f"Channel type. Use Source directly (ADR 0011)."
```

8. assistant (2026-07-06T09:09:36.490Z)

The finding is confirmed by reading the code. Let me verify with a repro to be adversarial. The key question: does the gate at line 190 let a brand-new channel through, and does `upsert\_channel` then set `watched=True`?

Let me check `upsert\_source` to confirm whether it preserves `watched=True` on insert (vs. respecting a DEFAULT 0).

9. user (2026-07-06T09:09:37.243Z)

```
1104     async def upsert_source(conn: aiomysqlite.Connection, source: Source) -> Source:
1105         """Insert a source row, or update title/watched on conflict.
1106
1107         The conflict key is `(account_id, provider, provider_source_id)` per the
1108         ADR 0011 schema. Returns the persisted row.
1109         """
1110         await conn.execute(
1111             """INSERT INTO sources
1112                 (account_id, provider, provider_source_id, title, kind, watched,
1113                  seen_first_at, last_seen_at)
1114             VALUES (?, ?, ?, ?, ?, ?, ?, ?)
1115             ON CONFLICT(account_id, provider, provider_source_id) DO UPDATE SET
1116                 title=excluded.title,
1117                 watched=excluded.watched,
1118                 last_seen_at=excluded.last_seen_at""",
1119             (
1120                 source.account_id,
1121                 source.provider,
1122                 source.provider_source_id,
1123                 source.title,
1124                 source.kind,
1125                 int(source.watched),
1126                 source.seen_first_at.isoformat(),
1127                 (source.last_seen_at.isoformat() if source.last_seen_at else None),
1128             ),
1129         )
1130         await conn.commit()
1131         conn.row_factory = aiomysqlite.Row
1132         async with conn.execute(
1133             """SELECT * FROM sources
1134                 WHERE account_id = ? AND provider = ? AND provider_source_id = ?""",
1135             (source.account_id, source.provider, source.provider_source_id),
1136         ) as cur:
1137             row = await cur.fetchone()
1138             assert row is not None # just upserted by the conflict key
1139             return _row_to_source(row)
1140
1141
1142     async def get_source_by_provider_id(
1143         conn: aiomysqlite.Connection, account_id: int, provider: str, provider_source_id: str
```

10. assistant (2026-07-06T09:09:44.006Z)

Confirmed. `upsert\_source` explicitly inserts `int(source.watched)` = 1, overriding the schema `DEFAULT 0`. And `upsert\_channel` hard-codes `watched=True`.

Now the critical adversarial check: is this within THIS PR's diff? The finding cites `telethon\_client.py:189-190` (the gate) as new in this PR. But `upsert\_channel`'s `watched=True` shim (db.py:1402) and `upsert\_source` are labeled ADR 0011 ? I need to confirm whether the gate is actually in this PR's diff and whether the deviation is attributable to this PR's code. Let

me check the diff.

11. user (2026-07-06T09:09:45.493Z)

```
db87cc0 feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR
0009, Track E)
35f4f70 feat(commands): extract CommandService ? provider-independent command execution (ADR
0011, Track D) (#38)
12d21b9 feat(delivery): destinations, routing_rules, delivery_attempts (ADR 0011, Track C) (#37)
---DIFF telethon---
commit db87cc02915f6343744a6c5c99f84aafd8418da5
Author: Avi Dullu <avi.dullu@gmail.com>
Date: Mon Jul 6 14:31:25 2026 +0530
```

```
feat(scanning): selective source scanning ? /watch, /unwatch, /channels, /watchlist (ADR
0009, Track E)
```

```
diff --git a/src/tg_video_filter/telethon_client.py b/src/tg_video_filter/telethon_client.py
index 883e3f8..ebb7416 100644
--- a/src/tg_video_filter/telethon_client.py
+++ b/src/tg_video_filter/telethon_client.py
@@ -181,6 +181,15 @@ def make_new_message_handler(
     message_id = message.id
     document_id = _extract_document_id(message)

+    # --- watched gate (ADR 0009, Track E) -----
+    # Skip ingestion for unwatched sources. The channel must already be in
+    # sources (lazy-created by the /channels enumeration or the legacy
+    # upsert_channel below); if absent, we let the legacy path upsert it
+    # with watched=True for backwards-compatibility.
+    src = await db.get_source_by_provider_id(conn, user.id, "telegram", str(channel_id))
+    if src is not None and not src.watched:
+        return # silently skip unwatched source
+
     # --- three-tier dedup (cheapest first) ---

     # Tier 0: same (user, channel, message) instance ? already processed.
```

12. assistant (2026-07-06T09:09:50.262Z)

The gate at lines 184-191 is added by THIS PR's diff. The `if src is not None and not src.watched: return` is the new code. This is the exact line the finding targets.

Now the crux of adversarial verification: the finding claims the gate should treat absent rows as unwatched. The PR author's comment explicitly says the opposite is intentional ("if absent, we let the legacy path upsert it with watched=True for backwards-compatibility"). So this is a genuine deviation from ADR 0009 lines 130-131 ("New channels discovered after the migration default to watched = 0 and must be explicitly watched").

Let me run the repro to confirm the end-to-end behavior beyond doubt.

13. user (2026-07-06T09:11:58.372Z)

Command running in background with ID: bojycxffh. Output is being written to:
C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bojycxffh.output. You will be notified when it completes. To check interim output, use Read on that file path.

14. user (2026-07-06T09:12:04.486Z)

Traceback (most recent call last):

```
File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 21, in <module>
    asyncio.run(main())
```

```

~~~~~^
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
195, in run
    return runner.run(main)
~~~~~^

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line
118, in run
    return self._loop.run_until_complete(task)
~~~~~^

File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
725, in run_until_complete
    return future.result()
~~~~~^

File "C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-
indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\scratchpad\repro.py", line 9, in main
    acc = await db.upsert_account(conn, Account(provider="telegram", provider_account_id="u1",
display_name="U"))
~~~~~^
~~~~~^
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\pydantic\main.py", line 263, in
__init__
    validated_self = self.__pydantic_validator__.validate_python(data, self_instance=self)
pydantic_core._pydantic_core.ValidationError: 1 validation error for Account
id
  Field required [type=missing, input_value={'provider': 'telegram', ...1', 'display_name':
'U'}, input_type=dict]
  For further information visit https://errors.pydantic.dev/2.13/v/missing

```

15. assistant (2026-07-06T09:12:07.118Z)

Let me check the Account/upsert_account signature quickly.

16. user (2026-07-06T09:12:08.784Z)

```

936:async def upsert_account(conn: aiosqlite.Connection, account: Account) -> Account:
937-     """Insert or update an account row; returns the persisted Account."""
938-     await conn.execute(
939-         """INSERT INTO accounts (id, telegram_id, provider, display_name, created_at)
940-         VALUES (?, ?, ?, ?, ?)
941-         ON CONFLICT(id) DO UPDATE SET
942-             telegram_id=excluded.telegram_id,
943-             provider=excluded.provider,
944-             display_name=excluded.display_name""",
945-         (
946-             account.id,
947-             account.telegram_id,
948-             account.provider,
949-             account.display_name,
950-             account.created_at.isoformat(),

```

17. assistant (2026-07-06T09:12:12.399Z)

The Account model differs. Rather than fight the model constructors, the mechanism is already proven by direct code reading: `upsert\_source` inserts `int(source.watched)` and `upsert\_channel` passes `watched=True`. Let me just test the core mechanic directly with raw SQL to confirm the schema default and the upsert behavior.

18. user (2026-07-06T09:12:24.282Z)

```

Account fields: ['id', 'telegram_id', 'provider', 'display_name', 'created_at']
get_source_by_provider_id(-1005) before any traffic = None
PR gate `if src is not None and not src.watched: return` -> skips? False
after upsert_channel, watched = True
raw insert (no watched given) -> schema DEFAULT watched = 0
Exception in thread Thread-1 (_connection_worker_thread):

```

Traceback (most recent call last):

```
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 66, in
_connection_worker_thread
    future.get_loop().call_soon_threadsafe(set_result, future, result)
    ~~~~~^
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
878, in call_soon_threadsafe
    self._check_closed()
    ~~~~~^
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
556, in _check_closed
    raise RuntimeError('Event loop is closed')
RuntimeError: Event loop is closed
```

During handling of the above exception, another exception occurred:

Traceback (most recent call last):

```
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\threading.py", line 1044, in
_bootstrap_inner
    self.run()
    ~~~~~^
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\threading.py", line 995, in
run
    self._target(*self._args, **self._kwargs)
    ~~~~~^
File "C:\Users\avidu\Projects\Telegram\.venv\Lib\site-packages\aiosqlite\core.py", line 75, in
_connection_worker_thread
    future.get_loop().call_soon_threadsafe(set_exception, future, e)
    ~~~~~^
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
878, in call_soon_threadsafe
    self._check_closed()
    ~~~~~^
File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line
556, in _check_closed
    raise RuntimeError('Event loop is closed')
RuntimeError: Event loop is closed
```

19. assistant (2026-07-06T09:12:36.971Z)

The repro output (the trailing error is just an unclosed-connection teardown warning, irrelevant) confirms every link in the chain:

```
- `get_source_by_provider_id(-1005)` before traffic = `None`
- PR gate `if src is not None and not src.watched: return` ? skips? `False` (does NOT skip a
brand-new channel)
- after `upsert_channel`, `watched = True`
- schema `DEFAULT watched = 0` (so the ADR-intended default is being overridden by both the shim
and upsert_source)
```

Now the adversarial counter-check: is there anything *upstream* of this handler that would prevent a truly-new channel from reaching `on\_new\_message`? ADR line 108-110 mentions a destination-exclusion check "before the watched-flag check" ? but that's Track E routing-loop code which this PR does not implement (diff is only the gate). And the handler registers `events.NewMessage(incoming=True)` with no `chats=` filter (ADR line 10-11), so every channel the account is a member of reaches the handler. A new source with no row therefore falls straight through.

One more consideration for severity: the ADR's backwards-compat section (line 119-133) intends that *pre-existing* seen channels stay watched via the migration backfill (`UPDATE ... WHERE seen\_first\_at IS NOT NULL`), while *new* channels default to 0. The PR's gate instead treats "no row" as watched, which conflates new channels with legacy ones. This is a real deviation from the accepted ADR and defeats the core purpose of the feature ("scans nothing until they explicitly /watch").

The finding is CONFIRMED. Severity: the finding says blocker. Let me weigh honestly. The feature's entire stated purpose ? opt-in scanning so a freshly-onboarded user "scans nothing" ? is defeated for any not-yet-seen channel: it gets ingested and delivered on first traffic and permanently after. That is the feature being broken on its core promise on the live path, and it's the exact regression ADR 0009 was written to prevent. Blocker is justified.

20. user (2026-07-06T09:12:50.542Z)

Structured output provided successfully